

系统设计说明书

项目名称： 社交网络平台（Social Network Platform）

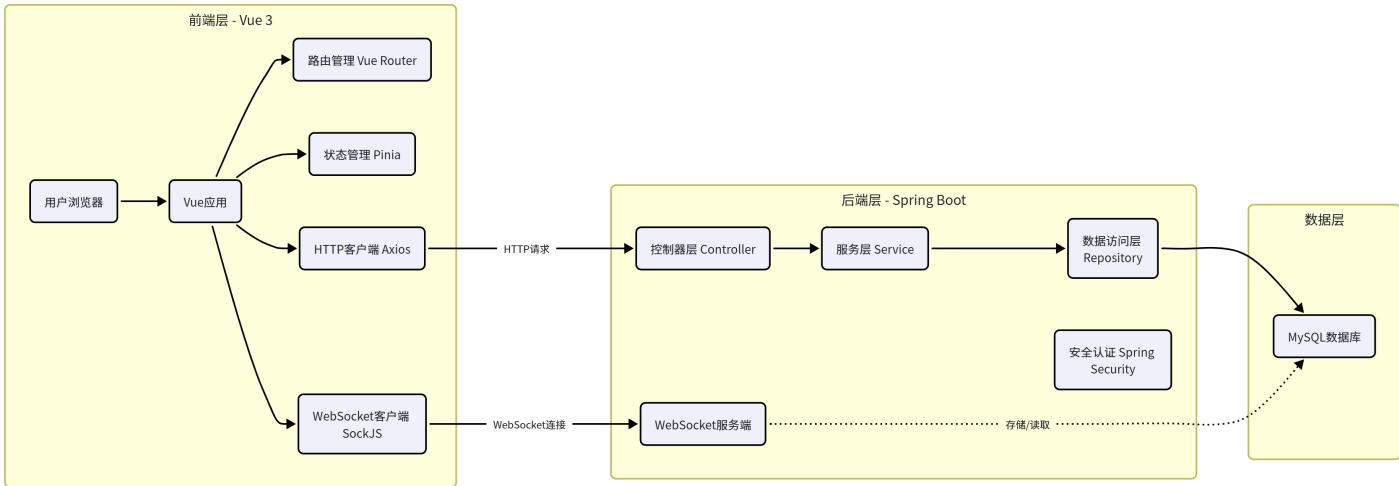
编制人： 向楚玉、胡紫怡

1. 系统体系架构

本系统采用基于 **B/S（浏览器/服务器）** 模式的**前后端分离架构**。

说明： 在开发与联调阶段，前端（Vue开发服务器）和后端（Spring Boot内置Tomcat）分别部署于两台不同的开发主机上，通过网络（校园网或WiFi热点）进行通信。前端通过后端主机的局域网IP地址（如 `http://192.168.x.x:8080`）调用后端API接口，构成完整的B/S（浏览器/服务器）访问链路。该部署方式与标准B/S架构完全一致，前后端分离部署，仅物理运行环境为开发阶段的临时网络环境。在项目演示时，可根据需要将前后端部署至同一台主机或迁移至云服务器，无需修改代码结构。

1.1 总体架构图



系统采用MVC体系结构风格，前端Vue对应视图层，后端Controller对应控制层，Service+Repository对应模型层

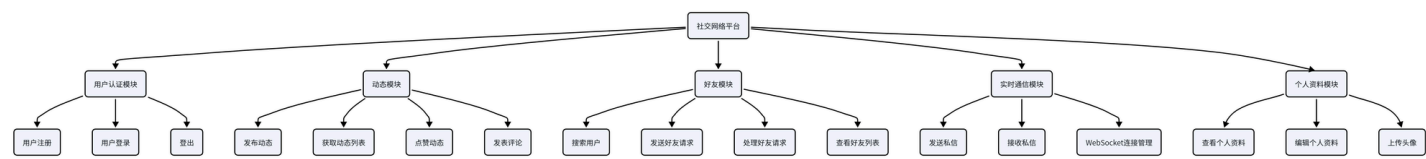
1.2 架构说明

层级	功能说明	技术选型
前端层	负责用户界面渲染、交互逻辑、路由控制与后端通信	Vue 3, Vite, Element Plus, Axios, SockJS

后端层	负责业务逻辑处理、数据持久化、身份认证与权限管理	Spring Boot, Spring Security, Spring Data JPA, WebSocket
数据层	负责数据的存储与管理	MySQL 8.0

2. 系统功能结构（层次结构）

系统功能从顶向下划分为四个层次：**表现层** → **业务层** → **数据层** → **基础设施层**。用户能直接接触的是表现层，而业务逻辑和数据存储则在业务层和数据层中完成，基础设施层则为整个系统提供运行支撑。

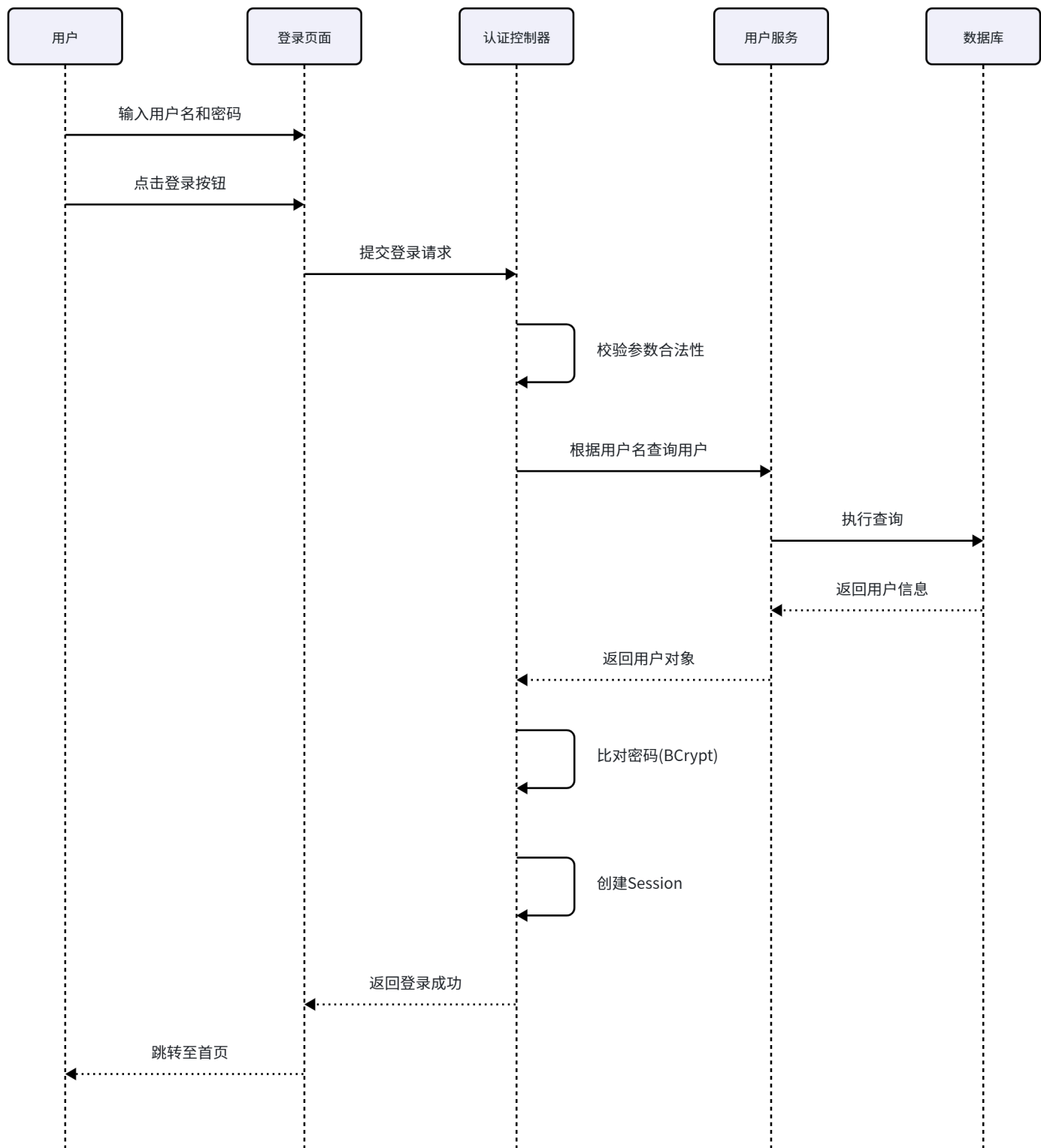


3. 核心用例的时序图（顺序图）及说明

这里选取“用户登录”、“发布动态”、“点赞动态”三个最核心的用例进行时序图建模。

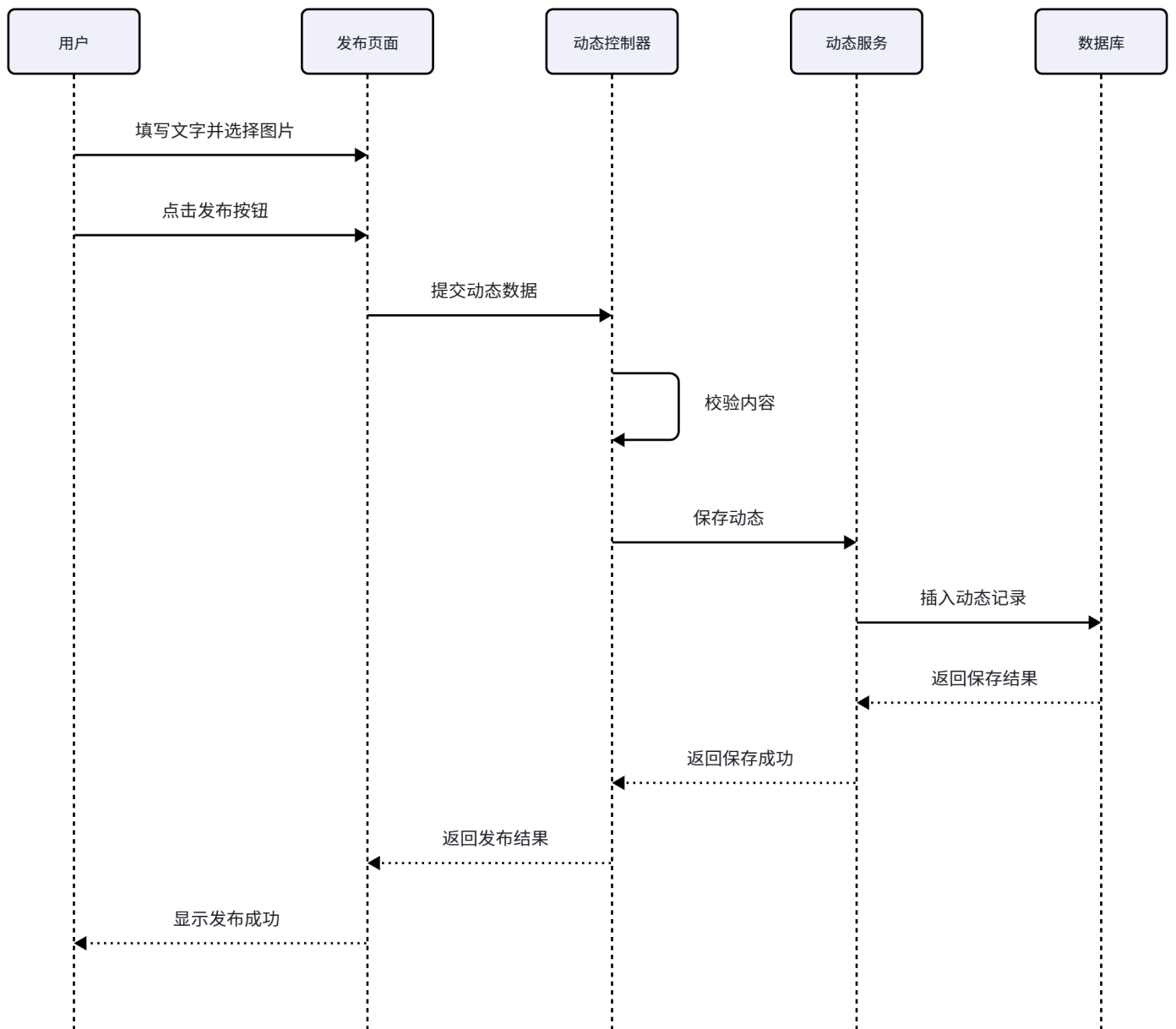
3.1 用户登录时序图

说明： 该时序图描述了用户通过账号密码登录系统的完整交互流程。用户提交凭证后，系统依次进行参数校验、用户查询、密码比对和会话创建，最终将登录结果返回给用户。



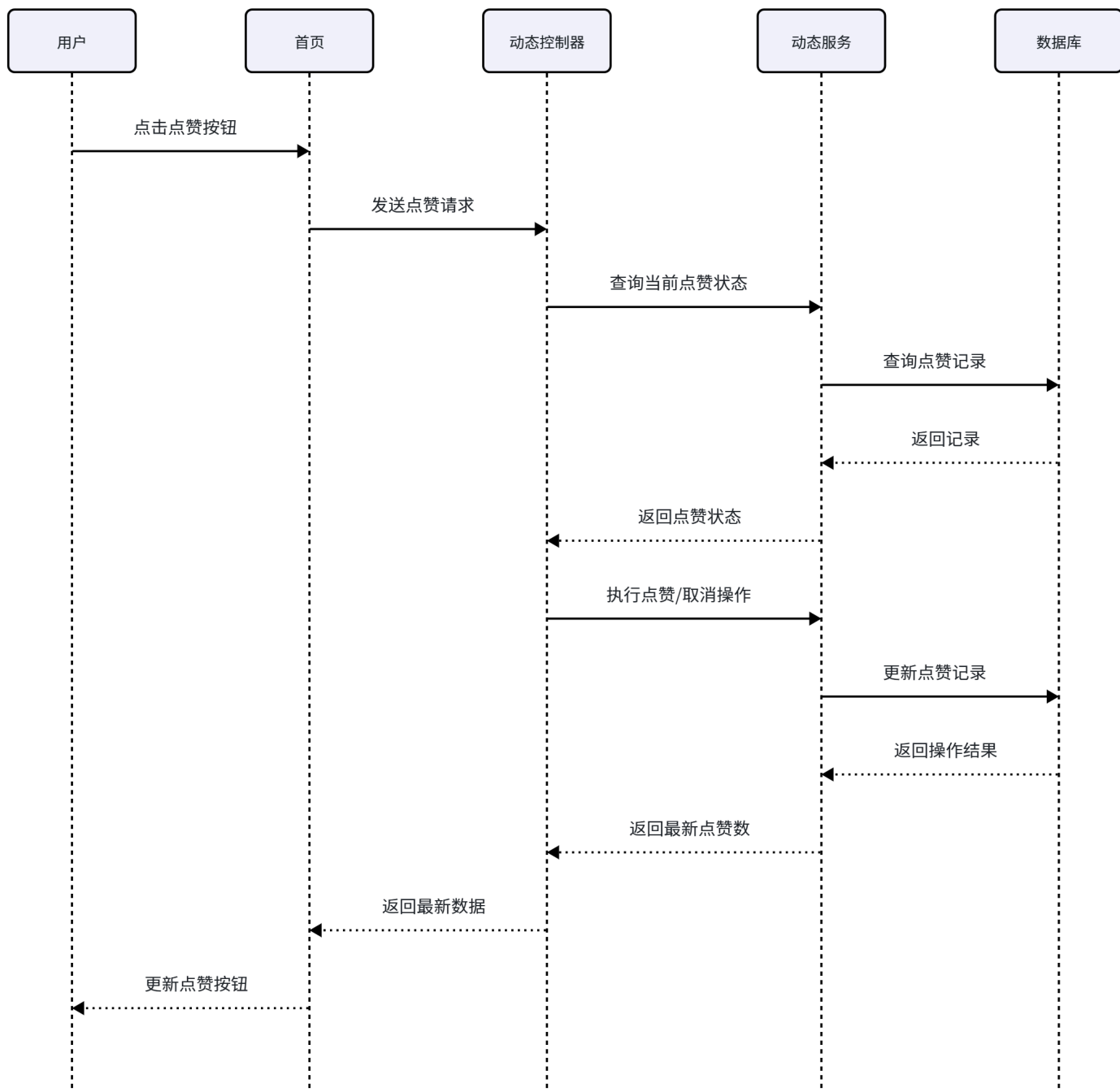
3.2 发布动态时序图

说明： 该时序图描述了用户发布一条新动态的完整流程。用户在发布页填写内容并提交后，系统先进行内容校验，然后将动态数据持久化到数据库，最后返回发布结果。



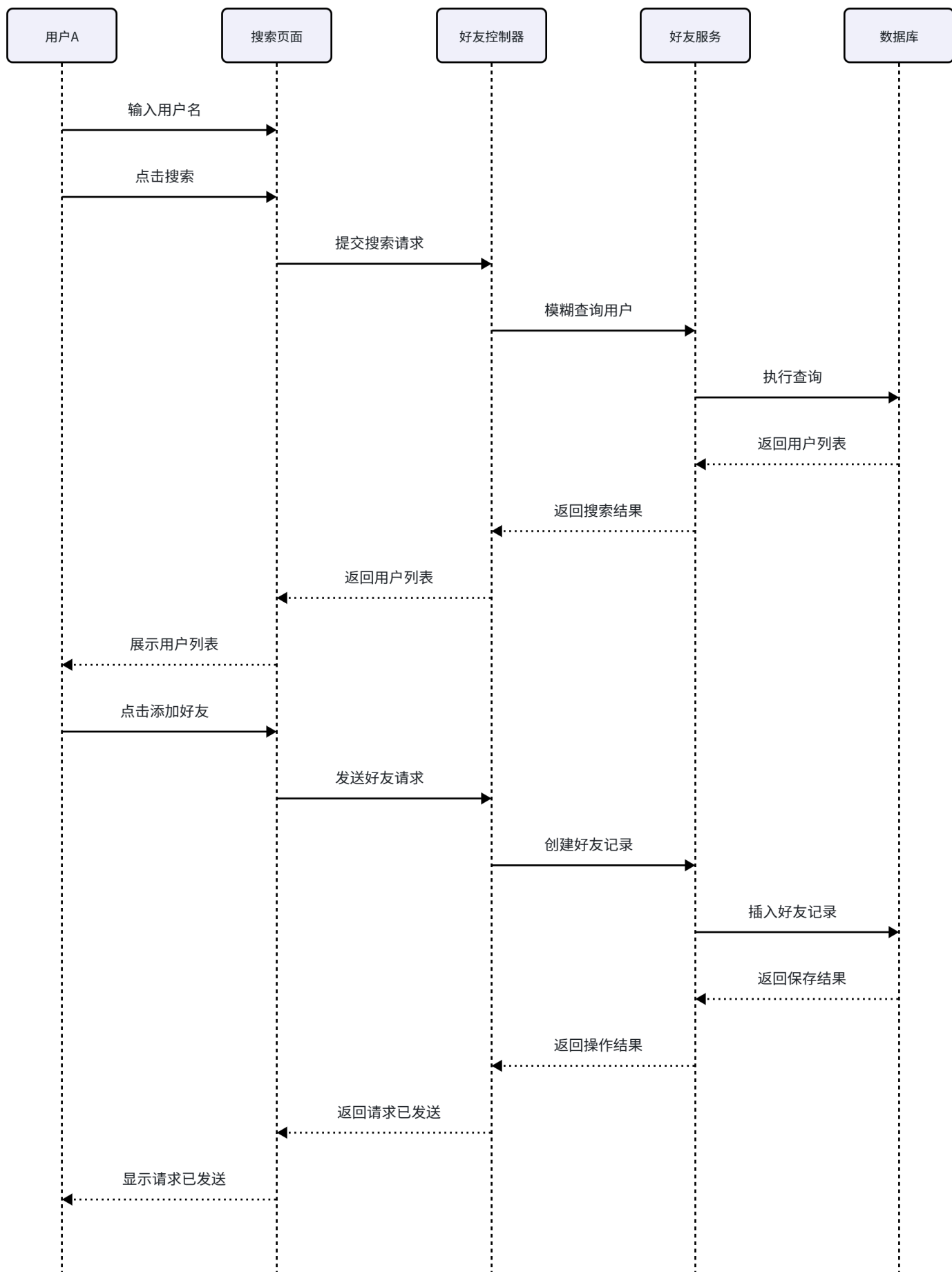
3.3 点赞动态时序图

说明： 该时序图描述了用户对某条动态进行点赞或取消点赞的操作流程。系统通过查询点赞记录判断当前状态，并执行相应的插入或删除操作，同时更新动态的点赞计数字段。



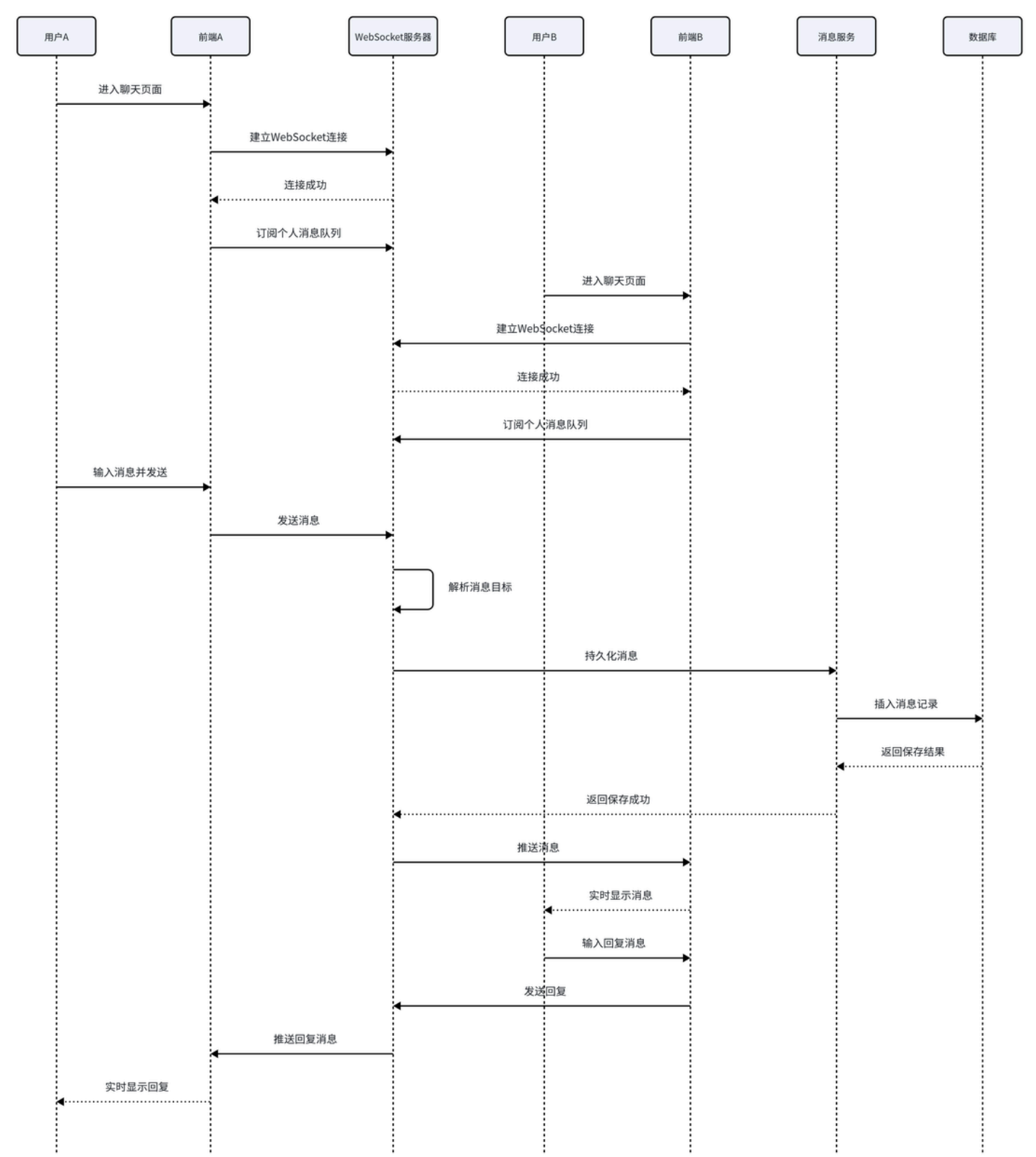
3.4 添加好友时序图

说明： 该时序图描述了用户向其他用户发起好友请求并得到处理的完整流程。首先发起方通过关键词搜索目标用户，然后发送好友请求，系统在数据库创建一条待确认的好友记录。接收方登录后查看请求列表，可以选择同意或拒绝，系统据此更新记录状态。



3.5 实时通信——建立WebSocket连接与收发私信时序图

说明： 该时序图描述了用户通过WebSocket建立实时通信连接，并完成私信发送与接收的完整过程。用户登录后，前端通过SockJS与后端建立WebSocket连接并订阅个人消息队列。当用户发送私信时，后端通过目标用户的队列将消息实时推送给接收方，实现即时通信。



时序图说明：

- 边界类（前端页面）：** 负责用户交互界面的展示，包括登录页、发布页、首页、聊天页面等，接收用户操作并将其转化为系统请求。

- **控制类（控制器）**：负责协调整个业务处理流程，包括认证控制器、动态控制器、好友控制器和 WebSocket 服务器，承担任务分发和流程控制的职责。
- **实体类（Service层）**：负责具体业务逻辑的实现和数据库操作，包括用户服务、动态服务、好友服务和消息服务。
- **数据库**：负责数据的持久化存储。

时序图中对象之间的消息传递，在实现时对应前端通过 Axios 发起的 HTTP 请求，以及后端内部 Controller、Service、Repository 三层之间的方法调用。

4. 复杂功能的算法设计

4.1 点赞/取消点赞算法（流程图）

点赞功能的核心逻辑是：先检查用户是否已经点过赞，如果已点赞则执行取消操作，否则执行点赞操作。算法通过一次数据库查询获取点赞记录，后续的插入或删除操作保证了数据的一致性。

开始



接收用户ID和动态ID



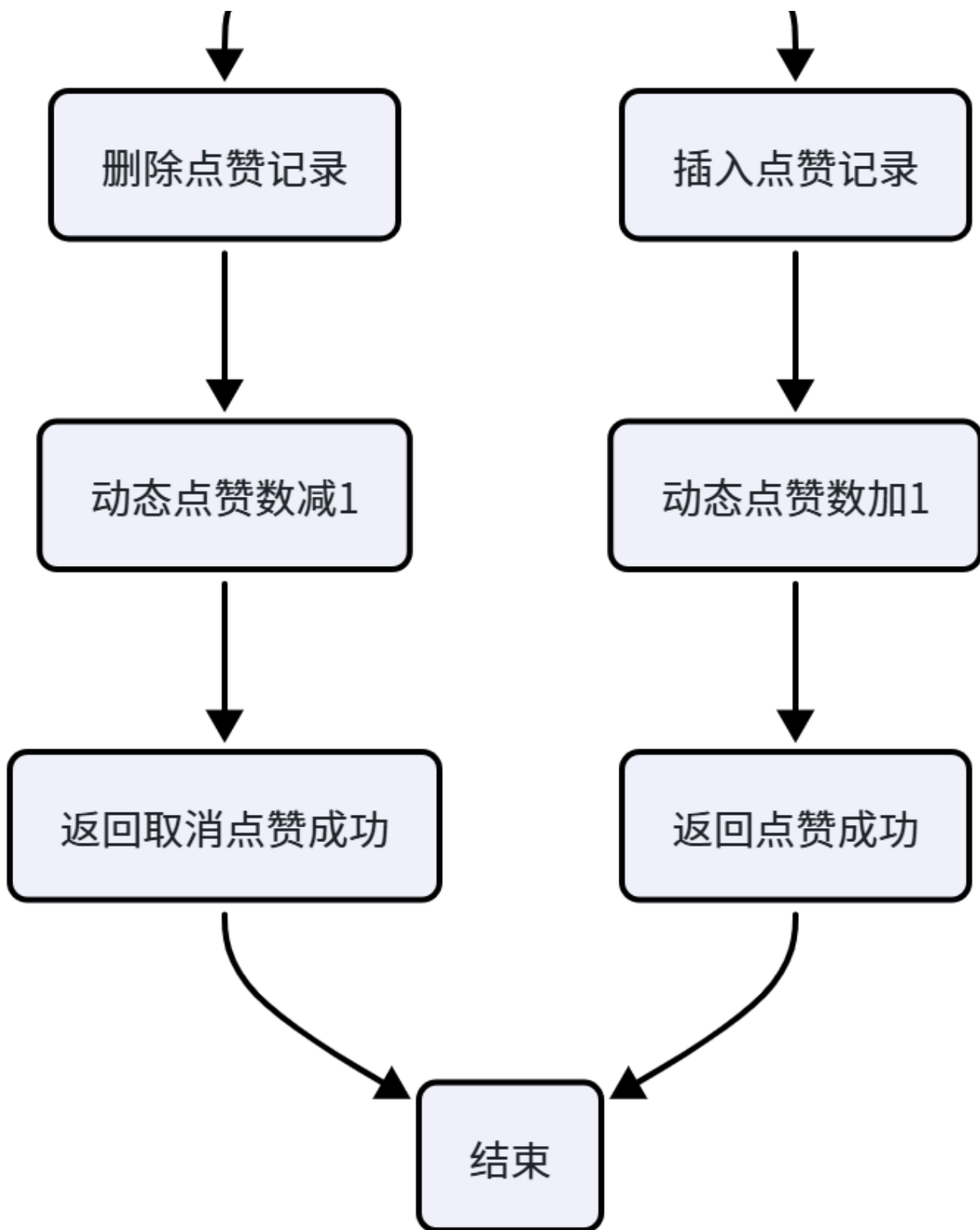
查询点赞记录是否存在



记录存在?

是

否



4.2 点赞/取消点赞算法（伪码）

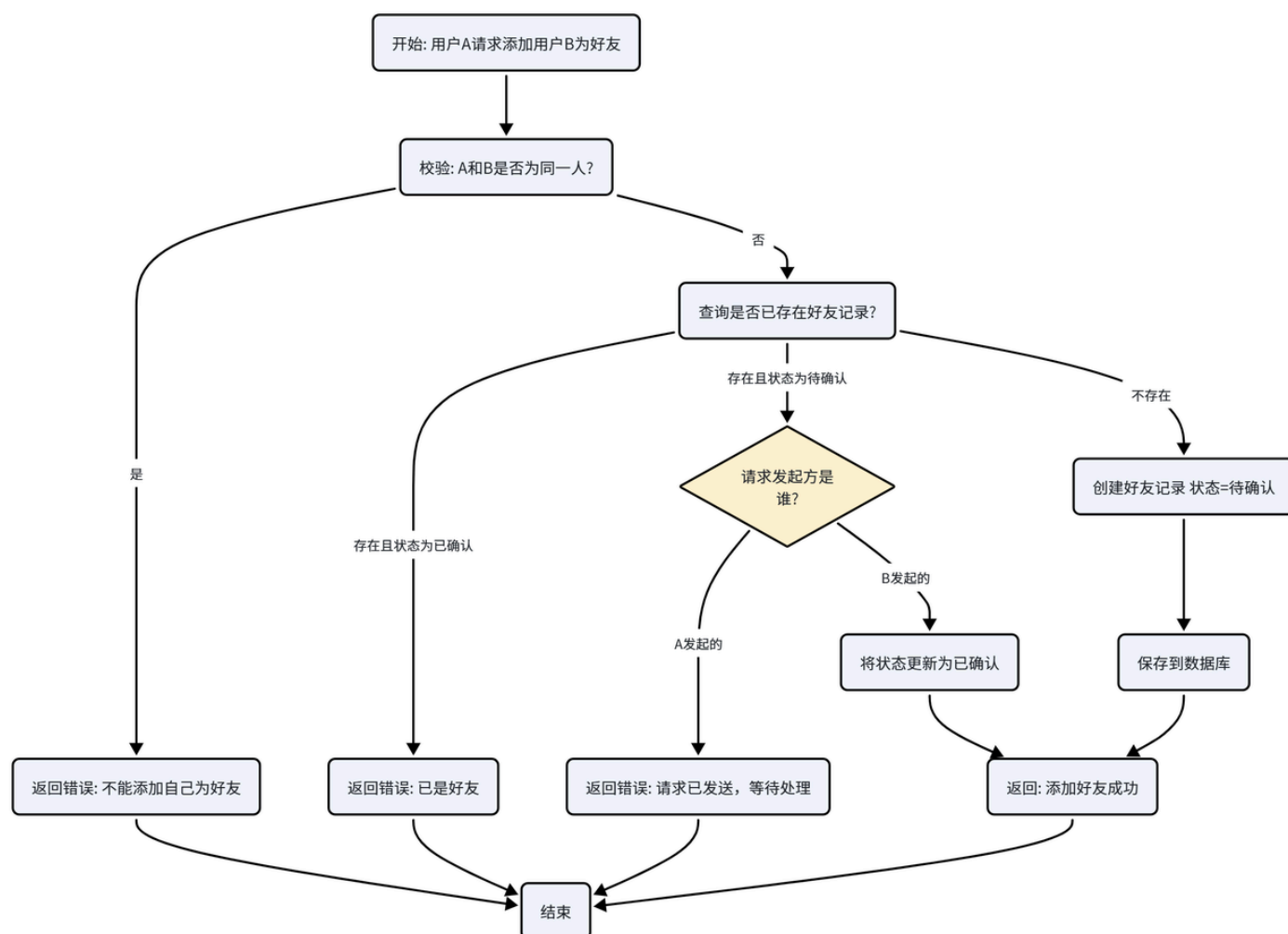
代码块

```
1  算法: toggleLike(userId, postId)
2  输入: 当前用户ID, 目标动态ID
3  输出: 操作结果 (点赞/取消点赞), 最新点赞数
4
5  1. 根据userId和postId查询点赞记录 likeRecord
```

```
6  2. 如果 likeRecord 存在:
7    2.1 删除该点赞记录
8    2.2 将动态的 likeCount 减1
9    2.3 返回 "取消点赞", likeCount
10 3. 否则:
11  3.1 创建新的点赞记录 (userId, postId)
12  3.2 将动态的 likeCount 加1
13  3.3 返回 "点赞成功", likeCount
```

4.3 好友请求处理算法（流程图）

好友请求处理需要确保请求的有效性，防止重复请求或自我请求。算法在创建好友记录前会进行多重校验，包括不能添加自己、不能重复发送、以及处理反向请求的自动通过逻辑。



4.4 好友请求处理算法（伪码）

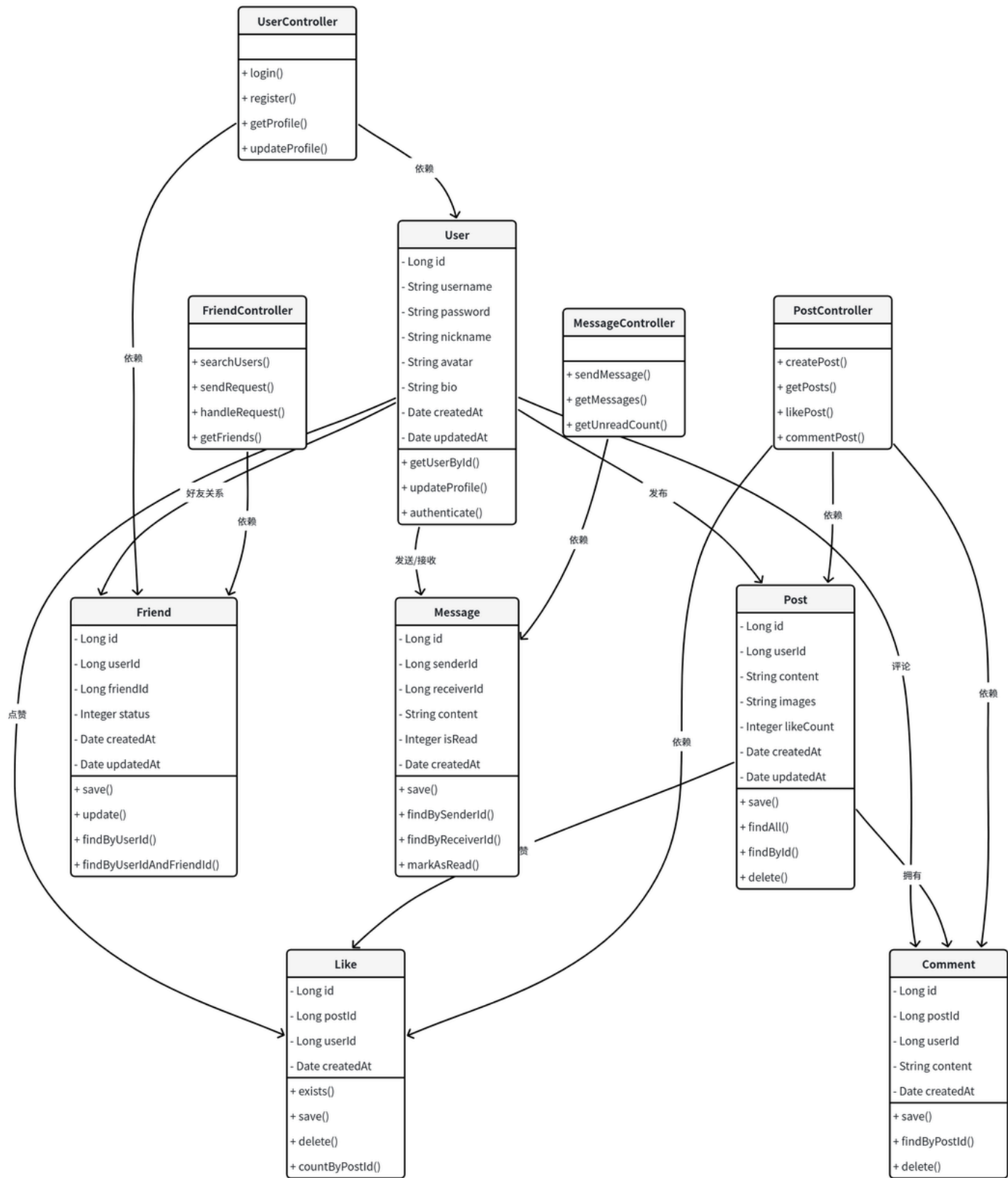
代码块

```
1  算法: sendFriendRequest(fromUserId, toUserId)
2  输入: 发起方用户ID, 接收方用户ID
```

```
3  输出：操作结果（成功/失败及原因）
4
5  1. 如果 fromUserId == toUserId:
6      返回 "不能添加自己为好友"
7  2. 查询 fromUserId 和 toUserId 之间的好友记录 friendRecord
8  3. 如果 friendRecord 存在:
9      3.1 如果 friendRecord.status == "已确认":
10         返回 "你们已经是好友了"
11     3.2 如果 friendRecord.status == "待确认":
12         如果 friendRecord.fromUserId == fromUserId:
13             返回 "好友请求已发送，请等待对方确认"
14         否则 (friendRecord.fromUserId == toUserId):
15             将 friendRecord.status 更新为 "已确认"
16             返回 "添加好友成功"
17 4. 否则 (friendRecord 不存在):
18     4.1 创建新的好友记录 (fromUserId, toUserId, 状态="待确认")
19     4.2 保存到数据库
20     4.3 返回 "好友请求已发送"
```

5. 面向对象方法类图的详细设计

5.1 系统核心类图



5.2 类图说明

类名	类类型	核心职责
User	实体类	表示系统用户，包含用户身份信息、个人资料。提供用户查询、资料更新和身份认证方法。

Post	实体类	表示用户发布的动态，包含内容、图片、点赞数等。提供动态的增删改查方法。
Like	实体类	表示用户对动态的点赞记录，用于判断点赞状态和统计点赞数。
Comment	实体类	表示用户对动态的评论，记录评论内容和发布者。
Friend	实体类	表示用户间的好友关系，包含状态字段（待确认/已确认）。
Message	实体类	表示用户间的私信消息，包含发送者、接收者、内容和已读状态。
UserController	控制类	处理用户认证和个人资料相关的HTTP请求。
PostController	控制类	处理动态发布、列表获取、点赞和评论相关的HTTP请求。
FriendController	控制类	处理好友搜索、请求发送、请求处理和好友列表获取相关的HTTP请求。
MessageController	控制类	处理消息发送、消息获取和未读消息统计相关的HTTP请求。

6. 接口设计

6.1 用户认证接口

接口	方法	请求体	响应格式	说明
/api/auth/register	POST	<pre>{"username":"xxx","password":"xxx"}</pre>	<pre>{"code":0,"message":"注册成功","data":null}</pre>	用户注册
/api/auth/login	POST	<pre>{"username":"xxx","password":"xxx"}</pre>	<pre>{"code":0,"message":"登录成功","data":{"id":1,"username":"xxx","nickname":"xxx"}}</pre>	用户登录
/api/auth/logout	POST	无	<pre>{"code":0,"message":"登出成功","data":null}</pre>	用户登出

6.2 动态接口

接口	方法	请求体	响应格式	说明
/api/posts	GET	无	<pre>{"code":0,"message":"success","data":[]}</pre>	获取动态列表（按时间倒序）

			<pre>": [{"id":1,"userId": 1,"username":"xxx" ,"content":"xxx"," images": [],"likeCount":5, "commentCount":2," createdAt":"2026- 07-08 10:00:00"}]}</pre>	
<code>/api/posts</code>	POST	<pre>{"content":"xxx", "images": ["url1","url2"]}</pre>	<pre>{"code":0,"messag e":"发布成 功","data": {"id":1,"createdAt ":"..."}}</pre>	发布动态
<code>/api/posts/{postId}/like</code>	POST	无	<pre>{"code":0,"messag e":"点赞成 功","data": {"liked":true,"lik eCount":6}}</pre>	点赞/取消点赞 (toggle)
<code>/api/posts/{postId}/comments</code>	POST	<pre>{"content":"xxx" }</pre>	<pre>{"code":0,"messag e":"评论成 功","data": {"id":1,"content": "xxx","createdAt": "..."}]}</pre>	发表评论
<code>/api/posts/{postId}/comments</code>	GET	无	<pre>{"code":0,"messag e":"success","data ": [{"id":1,"userId": 1,"username":"xxx" ,"content":"xxx"," createdAt":"..."}] }</pre>	获取评论列表

6.3 好友接口

接口	方法	请求体	响应格式	说明
----	----	-----	------	----

<code>/api/users/search</code>	GET	无 (Query参数: <code>keyword</code>)	<code>{"code":0,"message":"success", "data": [{"id":1,"username":"xxx","nickname":"xxx","avatar":"xxx"}] }</code>	按关键词搜索用户
<code>/api/friends/requests</code>	POST	<code>{"friendId":2}</code>	<code>{"code":0,"message":"好友请求已发送", "data":null }</code>	发送好友请求
<code>/api/friends/requests</code>	GET	无	<code>{"code":0,"message":"success", "data": [{"id":1,"userId":1,"friendId":2,"status":0,"createdAt":"..."}]}</code>	获取好友请求列表
<code>/api/friends/requests/{requestId}</code>	PUT	<code>{"status":1}</code>	<code>{"code":0,"message":"操作成功", "data":null }</code>	处理好友请求 (1-同意, 2-拒绝)
<code>/api/friends</code>	GET	无	<code>{"code":0,"message":"success", "data": [{"id":2,"username":"xxx","nickname":"xxx","avatar":"xxx"}] }</code>	获取好友列表

6.4 WebSocket消息接口

端点	协议	说明
<code>/ws/chat</code>	WebSocket (SockJS)	WebSocket连接端点

/app/chat	STOMP	发送消息目的地
/user/queue/messages	STOMP	用户个人消息队列（订阅）

消息格式：

代码块

```
1  // 发送消息
2  {
3      "receiverId": 2,
4      "content": "你好! "
5  }
6
7  // 接收消息
8  {
9      "id": 1,
10     "senderId": 1,
11     "senderUsername": "xxx",
12     "content": "你好! ",
13     "createdAt": "2026-07-08T10:00:00",
14     "isRead": false
15 }
```

备注： 本文档内容根据项目开发进度持续完善，部分接口可能存在调整，最终实现以实际代码为准。探索性功能（如WebSocket）可能根据实际情况选择性实现。

7. 数据库物理设计

7.1 物理设计总览

表名	说明	存储引擎	字符集	预估数据量
user	用户信息表	InnoDB	utf8mb4	1000条
post	动态表	InnoDB	utf8mb4	5000条
comment	评论表	InnoDB	utf8mb4	10000条
like	点赞记录表	InnoDB	utf8mb4	20000条
friend	好友关系表	InnoDB	utf8mb4	3000条
message	私信表	InnoDB	utf8mb4	10000条

7.2 表结构详细设计

(1) user（用户表）

字段名	数据类型	约束	说明
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	用户唯一标识
username	VARCHAR(20)	NOT NULL, UNIQUE	用户名
password	VARCHAR(100)	NOT NULL	BCrypt加密存储
nickname	VARCHAR(20)	DEFAULT NULL	用户昵称
avatar	VARCHAR(255)	DEFAULT NULL	头像URL
email	VARCHAR(100)	DEFAULT NULL	邮箱
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	注册时间
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE	更新时间

(2) post（动态表）

字段名	数据类型	约束	说明
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	动态唯一标识
user_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	发布者ID
content	VARCHAR(1000)	NOT NULL	文字内容
images	VARCHAR(2000)	DEFAULT NULL	图片URL列表（JSON数组）
likes_count	INT	DEFAULT 0	点赞总数
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	发布时间
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE	更新时间

(3) comment（评论表）

--	--	--	--

字段名	数据类型	约束	说明
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	评论唯一标识
post_id	BIGINT	NOT NULL, FOREIGN KEY → post(id)	所属动态ID
user_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	评论者ID
content	VARCHAR(200)	NOT NULL	评论内容
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	评论时间

(4) like（点赞记录表）

字段名	数据类型	约束	说明
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	点赞记录唯一标识
post_id	BIGINT	NOT NULL, FOREIGN KEY → post(id)	被点赞动态ID
user_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	点赞用户ID
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	点赞时间

约束： (user_id, post_id) 组合唯一，确保同一用户对同一条动态只产生一条点赞记录。

(5) friend（好友关系表）

字段名	数据类型	约束	说明
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	好友记录唯一标识
user_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	发起方用户ID
friend_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	接收方用户ID
status	TINYINT	NOT NULL, DEFAULT 0	状态：0待确认、1已确认、2已拒绝
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	请求发起时间
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE	状态更新时间

约束： (user_id, friend_id) 组合唯一。

(6) message（私信表）

字段名	数据类型	约束	说明
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	消息唯一标识
sender_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	发送者ID
receiver_id	BIGINT	NOT NULL, FOREIGN KEY → user(id)	接收者ID
content	VARCHAR(500)	NOT NULL	消息内容
is_read	TINYINT	DEFAULT 0	是否已读：0未读、1已读
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	发送时间

7.3 索引设计

表名	索引字段	索引类型	说明
user	username	UNIQUE	用户名唯一索引，加速登录查询
post	user_id	INDEX	加速按用户查询动态
post	created_at	INDEX	加速按时间排序获取动态列表
comment	post_id	INDEX	加速按动态查询评论
like	(user_id, post_id)	UNIQUE	防止重复点赞
like	post_id	INDEX	加速按动态统计点赞数
friend	(user_id, friend_id)	UNIQUE	防止重复好友记录
friend	user_id	INDEX	加速按用户查询好友列表
friend	friend_id	INDEX	加速按好友查询反向关系
message	sender_id	INDEX	加速按发送者查询消息
message	receiver_id	INDEX	加速按接收者查询消息

说明： 本设计为项目初期规划，实际开发中可能根据需求变化或技术实现情况进行表结构、字段、约束的调整，最终以代码实际落地为准。其中 `message` 表属于探索性功能，存在变动的可能。

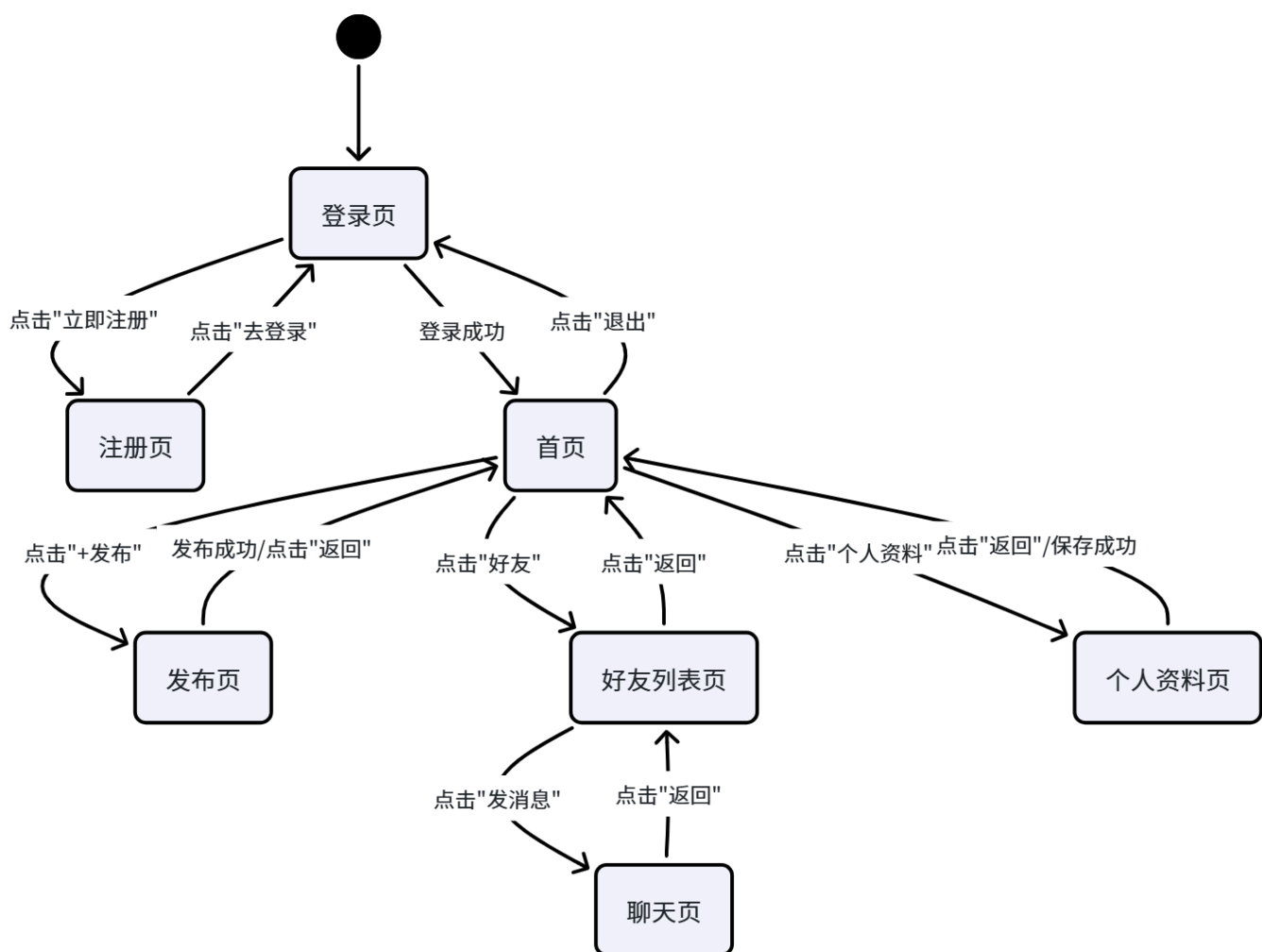
8. UI（界面）设计

8.1 页面清单

页面	路由	功能描述	访问权限
登录页	/login	用户登录入口，含用户名/密码表单和注册跳转	游客
注册页	/register	用户注册入口，含用户名/密码/确认密码表单	游客
首页（信息流）	/home	展示所有动态列表，支持点赞和评论	已登录用户
发布动态页	/publish	发布新动态，支持文字输入和图片上传	已登录用户
个人资料页	/profile	查看和编辑个人资料	已登录用户
好友列表页	/friends	查看好友列表和好友请求	已登录用户
搜索用户页	/friends/search	按关键词搜索用户并发送好友请求	已登录用户
聊天页	/chat	好友间实时私信聊天	已登录用户

8.1.1 页面跳转关系图

下图展示了用户在主要页面之间的跳转路径，箭头方向表示用户操作触发的跳转方向。



说明：这张图帮助理解系统的主线流程。每个节点代表一个独立页面，箭头上的文字是触发跳转的用户操作。未登录用户只能访问登录页和注册页，所有敏感操作（发布动态、查看好友、聊天）都要求用户处于登录状态。

8.2 核心页面线框图描述

(1) 登录页

- **布局：**居中卡片式设计，背景为渐变色
- **核心元素：**Logo/标题、用户名输入框、密码输入框、登录按钮、注册跳转链接
- **交互反馈：**表单验证提示、登录失败弹窗、登录成功跳转首页

(2) 首页（动态列表）

- **布局：**顶部导航栏 + 动态列表（瀑布流式）
- **核心元素：**应用Logo/标题、发布按钮、退出按钮、动态卡片（头像、用户名、发布时间、内容、图片、点赞按钮、评论按钮）

- **交互反馈：**点赞即时更新、点击评论展开评论框、点击发布跳转发布页

(3) 发布动态页

- **布局：**顶部导航栏（返回按钮+发布按钮）+ 内容编辑区 + 图片上传区
- **核心元素：**文字输入框、图片上传按钮、已选图片预览、发布按钮
- **交互反馈：**字数统计、图片预览与删除、发布成功后跳转首页

(4) 个人资料页

- **布局：**顶部导航栏 + 资料卡片
- **核心元素：**头像（可点击上传）、昵称、用户名、个人简介、编辑按钮
- **交互反馈：**点击编辑进入可编辑状态、保存后即时更新

8.3 响应式设计策略

屏幕尺寸	适配策略	说明
桌面端（>1024px）	内容居中，最大宽度600px	模拟移动端体验，视觉聚焦
平板端（768-1024px）	内容宽度自适应，保持60%-80%	充分利用屏幕空间
移动端（<768px）	内容宽度100%，全屏显示	贴合手机使用习惯

8.4 配色方案

颜色用途	色值	说明
主色	#667eea → #764ba2	渐变主色，用于按钮和标题
背景色	#f5f7fa	页面背景
卡片背景	#ffffff	卡片和表单背景
主文字	#1f2a3a	正文颜色
次要文字	、	